

PyTorch & TensorFlow: Complete Theoretical Guide

Table of Contents

1. Tensors: The Foundation
 2. Neural Network Layers
 3. Activation Functions
 4. Loss Functions
 5. Optimizers
 6. Regularization Techniques
 7. Architectures Explained
 8. Training Concepts
-

Tensors: The Foundation

What are Tensors?

Definition: A tensor is a multi-dimensional array that generalizes scalars, vectors, and matrices to higher dimensions.

Mathematical Hierarchy:

- **Scalar (0D):** Single number $\rightarrow (5)$
- **Vector (1D):** Array of numbers $\rightarrow ([1, 2, 3])$
- **Matrix (2D):** 2D array $\rightarrow ([[1, 2], [3, 4]])$
- **Tensor (3D+):** Multi-dimensional array $\rightarrow ([[[...]]])$

Why Tensors?

Neural networks process data in batches and require efficient parallel computation. Tensors allow:

1. **Batch Processing:** Process multiple samples simultaneously
 - Example: Shape $(32, 784) = 32$ images of 784 pixels each
2. **GPU Acceleration:** GPUs are optimized for tensor operations
 - Matrix multiplication on GPU is 100-1000x faster than CPU
3. **Automatic Differentiation:** Track computational graphs for backpropagation
 - Frameworks automatically compute gradients

When to Use Different Tensor Operations?

Reshape vs View (PyTorch):

- **View:** Fast but requires contiguous memory. Use for simple reshaping
- **Reshape:** More flexible, creates copy if needed. Use when unsure about memory layout
- **Practical:** Use `reshape` by default, it's safer

Element-wise vs Matrix Operations:

- **Element-wise** (`(*)`): Multiplies corresponding elements. Use for scaling/masking
- **Matrix multiplication** (`(@)`): Dot product. Use for linear transformations in neural networks

Example:

```
python

# Element-wise: scaling features
scaled = features * weights # [batch, features] * [batch, features]

# Matrix multiplication: linear layer
output = features @ weights.T # [batch, in_features] @ [out_features, in_features]
```

Neural Network Layers

1. Linear/Dense Layer

Mathematical Definition:

```
y = xW^T + b
where:
- x: input [batch_size, in_features]
- W: weights [out_features, in_features]
- b: bias [out_features]
- y: output [batch_size, out_features]
```

What it does: Performs a linear transformation - every input neuron connects to every output neuron.

When to use:

- **Transformers and modern architectures:** Preferred over Adam
- **Better generalization:** Separates weight decay from optimization
- **Same as Adam otherwise:** Easy drop-in replacement

Adam vs AdamW:

- Adam: Weight decay via L2 regularization (couples with adaptive learning rate)
- AdamW: Decoupled weight decay (more effective)
- **Use AdamW** for better results, especially large models

4. RMSprop

Mathematical Definition:

$$v_t = \beta * v_{t-1} + (1-\beta) * \nabla L^2$$
$$\theta_{t+1} = \theta_t - \eta * \nabla L / (\sqrt{v_t} + \epsilon)$$

Similar to Adam but without momentum term

When to use:

- **RNNs:** Historically good for recurrent networks
 - **Alternative to Adam:** Sometimes works when Adam doesn't
 - **Simpler than Adam:** Fewer hyperparameters
-

Regularization Techniques

Regularization prevents overfitting by constraining model complexity.

1. L2 Regularization (Weight Decay)

Mathematical Definition:

$$\text{Loss_total} = \text{Loss_original} + \lambda * \Sigma(w^2)$$

$$\text{Gradient: } \nabla L_{\text{total}} = \nabla L + 2\lambda w$$

$$\text{Update: } w \leftarrow w - \eta(\nabla L + 2\lambda w) = (1-2\eta\lambda)w - \eta\nabla L$$

Effect: Weights decay toward zero

When to use:

- **Overfitting detected:** Large weights, poor generalization
- **Many parameters:** Penalizes model complexity
- **Default regularization:** Common choice with dropout

Why it works:

- **Ockham's razor:** Simpler models generalize better
- **Smaller weights:** Less sensitive to individual features
- **Smooth predictions:** Prevents over-reliance on single features

Practical example:

```
python

#  $\lambda$  (weight_decay) values:
# 0.0001 - 0.001: Light regularization
# 0.01 - 0.1: Heavy regularization

optimizer = Adam(lr=0.001, weight_decay=0.01)

# Effect on large weight:  $w=10$ 
# Without L2:  $w$  might grow unbounded
# With L2:  $w$  penalized, shrinks toward 0
```

2. L1 Regularization

Mathematical Definition:

$$\text{Loss_total} = \text{Loss_original} + \lambda * \sum |w|$$

Effect: Drives weights exactly to zero (sparse)

When to use:

- **Feature selection:** Want to know which features important
- **Sparse models:** Many weights should be zero
- **Interpretability:** Smaller number of active features

L1 vs L2:

- **L1:** Creates sparse weights (many exactly 0)
- **L2:** Creates small weights (none exactly 0)
- **Choose L1 when:** Need interpretability, feature selection
- **Choose L2 when:** Want all features with small influence

3. Dropout (Covered Earlier)

Main points:

- Randomly drops neurons during training
- Rate 0.5 for dense layers, 0.1-0.2 for conv layers
- Prevents co-adaptation

4. Early Stopping

Definition: Monitor validation loss during training. Stop when it stops improving.

When to use:

- **Always:** Simple, effective regularization
- **Unknown optimal epochs:** Let validation decide
- **Resource constraints:** Don't waste computation

How it works:

```
python

patience = 10 # Wait 10 epochs for improvement
best_val_loss = infinity

for epoch in range(max_epochs):
    train(...)
    val_loss = validate(...)

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        save_model()
        patience_counter = 0
    else:
        patience_counter += 1

    if patience_counter >= patience:
        break # Stop training

load_best_model() # Use best, not last
```

5. Data Augmentation

Definition: Create variations of training data (rotation, flip, crop, noise).

When to use:

- **Limited data:** < 1000 samples per class
- **Images:** Always beneficial
- **Overfitting:** Training acc >> validation acc

Common augmentations:

Images:

- Horizontal flip (for natural images)
- Rotation ($\pm 15^\circ$)
- Color jitter (brightness, contrast)
- Random crop
- Gaussian noise

Text:

- Synonym replacement
- Back-translation
- Random insertion/deletion

Why it works:

- Artificially increases dataset size
 - Forces model to learn invariances
 - Reduces overfitting without reducing capacity
-

Architectures Explained

1. Feedforward Neural Network (MLP)

Architecture:

Input $\rightarrow$ Dense $\rightarrow$ Activation $\rightarrow$ Dense $\rightarrow$ Activation $\rightarrow$... $\rightarrow$ Output

When to use:

- **Tabular data:** Features in rows, samples in columns
- **Structured data:** No spatial/temporal structure
- **Baseline model:** Start here before complex architectures
- **Small datasets:** $< 10,000$ samples

Examples:

- House price prediction (features: size, location, age)

- Credit scoring (features: income, debt, history)
- Simple classification (iris flowers)

Why it might fail:

- No spatial awareness (use CNN for images)
- No temporal awareness (use RNN for sequences)
- Requires all features at once

2. Convolutional Neural Network (CNN)

Architecture:

Input → Conv → Activation → Pool → Conv → Pool → Flatten → Dense → Output

Standard pattern:

- Increase filters: $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$
- Decrease spatial size: $224 \rightarrow 112 \rightarrow 56 \rightarrow 28$
- End with global pooling or flatten

When to use:

- **Images:** Primary choice
- **Spatial data:** Any grid-like structure
- **Local patterns matter:** Edges, textures, shapes
- **Translation invariance:** Object anywhere in image

Examples:

- Image classification (cats vs dogs)
- Object detection (find cars in image)
- Semantic segmentation (pixel-wise labeling)
- Medical imaging (X-ray diagnosis)

Key design choices:

1. Depth (number of layers):

- Shallow (3-5 layers): Simple tasks, small images
- Medium (10-20): Standard (ResNet-18, VGG-16)
- Deep (50-150): Complex tasks, large datasets

2. Filter sizes:

- Start: 7×7 or 5×5 (capture larger patterns)
- Middle/End: 3×3 (standard, efficient)
- 1×1 : Channel mixing, dimension reduction

3. Architecture patterns:

- **VGG**: Stack 3×3 convs, simple but deep
- **ResNet**: Skip connections, very deep (50-152 layers)
- **Inception**: Multiple kernel sizes in parallel
- **EfficientNet**: Compound scaling (width, depth, resolution)

3. Recurrent Neural Network (RNN/LSTM/GRU)

Architecture:

Input sequence $\rightarrow$ Embedding $\rightarrow$ LSTM $\rightarrow$ LSTM $\rightarrow$ Dense $\rightarrow$ Output

When to use:

- **Sequential data**: Order matters
- **Variable length**: Different sequence lengths
- **Temporal dependencies**: Past influences future

Examples:

- Language modeling (predict next word)
- Machine translation (English $\rightarrow$ French)
- Speech recognition (audio $\rightarrow$ text)
- Time series forecasting (stock prices)
- Video analysis (action recognition)

Sequence types:

1. **One-to-One**: Standard feedforward (not RNN)
2. **One-to-Many**: Image captioning (image $\rightarrow$ sentence)
3. **Many-to-One**: Sentiment analysis (sentence $\rightarrow$ positive/negative)
4. **Many-to-Many (same length)**: Video classification (frame-by-frame)
5. **Many-to-Many (different length)**: Translation (English $\rightarrow$ French)

4. Encoder-Decoder (Seq2Seq)

Architecture:

Encoder: Input sequence $\rightarrow$ LSTM $\rightarrow$ Context vector

Decoder: Context vector $\rightarrow$ LSTM $\rightarrow$ Output sequence

When to use:

- **Different length input/output:** Translation, summarization
- **Sequence-to-sequence:** Transform one sequence to another

With attention:

- Decoder can "attend" to different encoder positions
- Handles long sequences better
- Foundation for Transformers

5. Autoencoder

Architecture:

Encoder: Input $\rightarrow$ Compress $\rightarrow$ Latent representation

Decoder: Latent $\rightarrow$ Reconstruct $\rightarrow$ Output

Loss: Reconstruction error (MSE between input and output)

When to use:

- **Dimensionality reduction:** Alternative to PCA
- **Anomaly detection:** High reconstruction error = anomaly
- **Denoising:** Train on noisy input, clean target
- **Feature learning:** Use latent representation

Variants:

- **Variational Autoencoder (VAE):** Probabilistic latent space
- **Denoising Autoencoder:** Robust feature learning
- **Sparse Autoencoder:** L1 regularization on latent

6. Transfer Learning

Concept: Use model pretrained on large dataset, adapt to your task.

When to use:

- **Small dataset:** < 1000 samples per class
- **Similar task:** Pretrained domain relevant to yours
- **Limited compute:** Training from scratch expensive

Approaches:

1. Feature extraction (freeze base):

```
python

base_model.trainable = False # Freeze weights
# Only train new layers
```

- **When:** Very small dataset (< 1000 samples)
- **Fast:** Only train small head

2. Fine-tuning (unfreeze some layers):

```
python

# Train new layers first
base_model.trainable = False
model.fit(...) # Train head

# Then fine-tune base
base_model.trainable = True
model.fit(..., lr=1e-5) # Small learning rate
```

- **When:** Medium dataset (1000-10,000 samples)
- **Better performance:** Adapts features to your domain

3. Full training (unfreeze all):

- **When:** Large dataset (> 10,000 samples)
- **Best performance:** If enough data

Popular pretrained models:

- **Images:** ResNet, VGG, EfficientNet, Vision Transformer
- **Text:** BERT, GPT, RoBERTa, T5
- **Source:** ImageNet (images), Wikipedia+BookCorpus (text)

Training Concepts

1. Batch Size

Definition: Number of samples processed before updating weights.

Types:

- **Batch Gradient Descent:** Entire dataset (slow, memory intensive)
- **Stochastic Gradient Descent:** Single sample (noisy, unstable)
- **Mini-batch:** 16-512 samples (standard)

When to use different sizes:

Small batches (16-32):

- **Limited memory:** Large images, large models
- **Regularization effect:** Noise helps generalization
- **Faster iteration:** See more updates per epoch

Large batches (128-512):

- **Stable gradients:** Less noise in updates
- **Better hardware utilization:** GPUs prefer large batches
- **Faster training:** Fewer gradient computations

Practical guidelines:

- Start with 32 or 64
- Increase if training unstable
- Decrease if out of memory
- Powers of 2 for GPU efficiency

2. Learning Rate

Definition: Step size for parameter updates.

Impact:

- **Too high:** Loss explodes, unstable training
- **Too low:** Slow convergence, gets stuck
- **Just right:** Smooth, fast convergence

Finding good learning rate:

1. Learning rate finder:

```
python

# Start small, increase exponentially
lrs = [1e-7, 1e-6, ..., 1e-1]
for lr in lrs:
    loss = train_one_batch(lr)
    plot(lr, loss)

# Choose lr where loss decreasing fastest
```

2. Common starting points:

- Adam: 1e-3 or 3e-4
- SGD: 1e-2 or 1e-1
- Fine-tuning: 1e-5 or 1e-4

3. Learning rate schedules:

Step decay:

```
python

# Decrease by factor every N epochs
lr = 0.1
epochs [0-30]: lr = 0.1
epochs [31-60]: lr = 0.01
epochs [61+]: lr = 0.001
```

Exponential decay:

```
python

lr_t = lr_0 * e(-λt)
# Smooth decrease over time
```

Cosine annealing:

```
python

lr_t = lr_min + 0.5(lr_max - lr_min)(1 + cos(πt/T))
# Decreases then increases (helps escape local minima)
```

Reduce on plateau:

```
python

# If validation loss doesn't improve for N epochs:
lr = lr * 0.1 # Reduce by factor
```

3. Gradient Clipping

Definition: Limit gradient magnitude to prevent explosion.

Mathematical:

```
If ||gradient|| > threshold:
    gradient = threshold * gradient / ||gradient||
```

When to use:

- **RNNs:** Very prone to exploding gradients
- **Very deep networks:** Gradients can grow exponentially
- **Unstable training:** Loss suddenly spikes

Typical values:

- Clip norm: 1.0 or 5.0
- Clip value: 0.5

4. Validation Strategy

Purpose: Estimate model performance on unseen data.

Train/Validation/Test Split:

```
Total data: 100%
├── Training: 70-80% (Train model)
├── Validation: 10-15% (Tune hyperparameters)
└── Test: 10-15% (Final evaluation)
```

K-Fold Cross-Validation:

```
Split data into K folds (e.g., 5)
For each fold:
    Train on K-1 folds
    Validate on 1 fold
Average performance across folds
```

When to use K-fold:

- Small datasets (< 10,000 samples)
- Want robust estimate
- Have computation time (trains K models)

5. Metrics

Classification:

Accuracy:

Accuracy = Correct predictions / Total predictions

Use when: Classes balanced

Don't use when: Imbalanced (99% negative, 1% positive)

Precision & Recall:

Precision = True Positives / (True Positives + False Positives)

"Of predicted positives, how many correct?"

Recall = True Positives / (True Positives + False Negatives)

"Of actual positives, how many found?"

F1-Score = $2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$

Harmonic mean of precision and recall

When to use:

- **High Precision:** Spam detection (false positives annoying)
- **High Recall:** Disease detection (false negatives dangerous)
- **F1-Score:** Balance both

Regression:

MAE (Mean Absolute Error):

Interpretable, same units as target

Use when: Want interpretable error

RMSE (Root Mean Squared Error):

$$\text{RMSE} = \sqrt{\text{MSE}}$$

Penalizes large errors more

Use when: Large errors particularly bad

R² (Coefficient of Determination):

$$R^2 = 1 - (\text{SS}_{\text{res}} / \text{SS}_{\text{tot}})$$

Range: $(-\infty, 1]$, 1 = perfect, 0 = baseline

Use when: Want relative performance measure

6. Overfitting vs Underfitting

Underfitting:

- **Symptom:** Both train and val accuracy low
- **Cause:** Model too simple, not enough training
- **Solution:**
 - More complex model (more layers/neurons)
 - Train longer
 - Remove regularization
 - Better features

Overfitting:

- **Symptom:** Train accuracy high, val accuracy low
- **Cause:** Model memorizes training data
- **Solution:**
 - More data
 - Data augmentation
 - Regularization (dropout, L2)
 - Simpler model
 - Early stopping

Sweet spot:

- Train and validation accuracy similar
- Both reasonably high
- Model generalizes well

7. Mixed Precision Training

Definition: Use 16-bit floats (half precision) instead of 32-bit for speed.

Benefits:

- **2x faster:** Half the memory bandwidth
- **2x memory:** Can use larger batch sizes
- **Minimal accuracy loss:** Careful handling maintains precision

When to use:

- **Large models:** Memory constrained
- **Modern GPUs:** V100, A100, RTX series with Tensor Cores
- **Production:** Faster inference

How it works:

- **Forward pass:** 16-bit (FP16)
 - **Gradients:** 16-bit
 - **Master weights:** 32-bit (FP32)
 - **Loss scaling:** Prevent underflow
-

Practical Workflow

Complete Training Pipeline

1. Data Preparation:

```
python
- Load data
- Split train/val/test
- Normalize/standardize
- Create DataLoader
- Setup augmentation
```

2. Model Architecture:

```
python
```

- Choose architecture (MLP/CNN/RNN)
- Number of layers
- Hidden sizes
- Activation functions
- Regularization (dropout)

3. Training Setup:

python

- Loss function (task-appropriate)
- Optimizer (start with Adam)
- Learning rate (1e-3 for Adam)
- Batch size (32-64)
- Number of epochs

4. Training Loop:

python

- Forward pass
- Compute loss
- Backward pass
- Update weights
- Log metrics

5. Monitoring:

python

- Plot train/val loss
- Check for overfitting
- Monitor metrics
- Save best model

6. Hyperparameter Tuning:

python

- Adjust learning rate
- Try different architectures
- Tune regularization
- Optimize batch size

7. Evaluation:

python

- Load best model
- Evaluate on test set
- Compute final metrics
- Analyze errors

Common Mistakes to Avoid

1. **Not shuffling data:** Model learns order
 2. **Not normalizing:** Features on different scales
 3. **Testing on training data:** Overly optimistic results
 4. **Ignoring class imbalance:** Accuracy misleading
 5. **Not using validation set:** Can't detect overfitting
 6. **Too complex model first:** Start simple
 7. **Not checking for data leakage:** Information from test in train
 8. **Forgetting model.eval():** Dropout/BatchNorm behave differently
-

Quick Decision Tree

What task?

Classification (categorical output):

- Tabular data → MLP
- Images → CNN
- Text → RNN/LSTM or Transformer
- Time series → LSTM/GRU

Regression (continuous output):

- Tabular data → MLP
- Images → CNN
- Sequential → LSTM/GRU

Architecture depth:

- < 1K samples → Shallow (3-5 layers)

- 1K-100K samples → Medium (10-20 layers)
-  100K samples → Deep (20-50+ layers) or pretrained

Regularization:

- Overfitting → Dropout (0.3-0.5), L2 (0.001-0.01)
- Small data → Data augmentation
- Always → Early stopping

Optimizer:

- Default → Adam (lr=1e-3)
- Not working → Try AdamW or SGD with momentum
- RNN → Try RMSprop

Loss function:

- Binary classification → BCELoss + Sigmoid
- Multi-class → CrossEntropyLoss + Softmax
- Regression → MSE or MAE
- Regression with outliers → Huber Loss

This guide should give you the theoretical foundation to understand when, why, and how to use each component effectively!

- **Classification tasks:** Final layer to produce class probabilities
- **Regression:** Output layer for continuous predictions
- **Feature transformation:** Hidden layers to learn representations
- **After flattening:** Convert CNN features to predictions

Why it works:

- **Universal function approximation:** Multiple linear layers with non-linearities can approximate any function
- **Feature combinations:** Learns weighted combinations of input features

Practical example:

```
python
```

Image classification: 28x28 image $\rightarrow$ 10 classes

Flatten: 784 inputs

Hidden: 128 neurons (learns representations)

Output: 10 classes

```
model = nn.Sequential(  
    nn.Linear(784, 128), # Learn 128 features from pixels  
    nn.ReLU(),  
    nn.Linear(128, 10) # Combine features for classification  
)
```

Parameters: $(in_features \times out_features + out_features)$ (weights + biases)

2. Convolutional Layers (Conv2D)

Mathematical Definition:

For each output position (i, j):

$$y[i,j] = \sum \sum x[i+m, j+n] * kernel[m, n] + bias$$

The kernel "slides" over the input, computing dot products.

What it does: Applies filters (kernels) that slide over the input to detect local patterns.

When to use:

- **Images:** Natural choice for spatial data
- **Time series:** Conv1D for temporal patterns
- **Any grid-like data:** Where local patterns matter

Why it works:

- **Translation invariance:** Same pattern detected anywhere in image
- **Parameter sharing:** Same kernel used across entire input (fewer parameters)
- **Hierarchy:** Early layers detect edges, later layers detect complex patterns

Key Concepts:

1. Kernel/Filter Size:

- 3×3 : Standard, captures local patterns (edges, textures)
- 5×5 or 7×7 : Larger receptive field, captures broader patterns
- 1×1 : Channel-wise mixing, reduces dimensions

2. Stride:

- Stride=1: Output nearly same size as input
- Stride=2: Downsamples by 2x (common alternative to pooling)
- Use **larger strides** to reduce computation and spatial dimensions

3. Padding:

- `padding='same'`: Output size = input size (adds zeros around border)
- `padding='valid'`: No padding, output shrinks
- Use **'same'** to preserve spatial dimensions

4. Number of Filters:

- Controls how many different patterns to detect
- Common: $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ (increases with depth)
- More filters = more capacity but more computation

Practical example:

```
python

# Image classification CNN
Conv2D(3, 32, kernel_size=3, padding=1) # 3 RGB → 32 features
# Detects edges, colors, simple textures

Conv2D(32, 64, kernel_size=3, padding=1) # 32 → 64 features
# Combines edges into shapes, patterns

Conv2D(64, 128, kernel_size=3, padding=1) # 64 → 128 features
# Detects complex objects, parts
```

Parameters: $\text{kernel_size}^2 \times \text{in_channels} \times \text{out_channels} + \text{out_channels}$

- Example: 3×3 kernel, $32 \rightarrow 64$ channels = $3 \times 3 \times 32 \times 64 + 64 = 18,496$ parameters

Why fewer parameters than Dense?

- Dense layer with same input ($32 \times 32 \times 32 = 32,768$ inputs) to 64 outputs: 2+ million parameters
- Conv layer: Only ~18K parameters but achieves spatial understanding

3. Pooling Layers

Mathematical Definition:

Max Pooling:

For each 2×2 region:
output = max(x[i:i+2, j:j+2])

Average Pooling:

For each 2×2 region:
output = mean(x[i:i+2, j:j+2])

What it does: Downsamples spatial dimensions by taking maximum or average over regions.

When to use:

- **After Conv layers:** Reduce spatial dimensions progressively
- **Reduce computation:** Fewer pixels = faster training
- **Prevent overfitting:** Adds translation invariance
- **Control output size:** Before dense layers

Max vs Average Pooling:

- **MaxPool:** Keeps strongest activations, better for textures/patterns
 - Use for: Classification, feature detection
- **AvgPool:** Smooths activations, better for general features
 - Use for: When preserving all information matters
- **Global Average/Max Pooling:** Reduce entire spatial dimension to 1×1
 - Use before final classification layer (replaces flatten)

Practical example:

```
python

# Input: 224×224 image
Conv2D(3, 64)    # 224×224×64
MaxPool2D(2)     # 112×112×64 (halved spatial dimensions)

Conv2D(64, 128)  # 112×112×128
MaxPool2D(2)     # 56×56×128

# Reduces computation by 75% at each step
# Final spatial size manageable for dense layers
```

Why use pooling?

- **Computational efficiency:** 2×2 pooling reduces dimensions by $4 \times$
 - **Invariance:** Small translations don't change output
 - **Receptive field:** Each neuron "sees" larger input region
-

4. Recurrent Layers (RNN, LSTM, GRU)

Mathematical Definitions:

Simple RNN:

$$h_t = \tanh(W_{ih} * x_t + b_{ih} + W_{hh} * h_{t-1} + b_{hh})$$

where:

- h_t : hidden state at time t
- x_t : input at time t
- h_{t-1} : previous hidden state

LSTM (Long Short-Term Memory):

$$\text{Forget gate: } f_t = \sigma(W_f * [h_{t-1}, x_t] + b_f)$$

$$\text{Input gate: } i_t = \sigma(W_i * [h_{t-1}, x_t] + b_i)$$

$$\text{Output gate: } o_t = \sigma(W_o * [h_{t-1}, x_t] + b_o)$$

$$\text{Cell state: } C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_c * [h_{t-1}, x_t] + b_c)$$

$$\text{Hidden state: } h_t = o_t \odot \tanh(C_t)$$

where σ = sigmoid, $\odot$ = element-wise multiplication

GRU (Gated Recurrent Unit):

$$\text{Reset gate: } r_t = \sigma(W_r * [h_{t-1}, x_t])$$

$$\text{Update gate: } z_t = \sigma(W_z * [h_{t-1}, x_t])$$

$$\text{Hidden state: } h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tanh(W * [r_t \odot h_{t-1}, x_t])$$

When to use:

RNN (Simple):

- **Short sequences:** < 20 timesteps
- **Simple patterns:** When long-term dependencies not critical
- **Fast training needed:** Fewer parameters than LSTM/GRU
- **Example:** Sentiment analysis on short sentences

LSTM:

- **Long sequences:** 100s to 1000s of timesteps
- **Long-term dependencies:** When context from far back matters
- **Language modeling:** Text generation, translation
- **Time series:** Stock prices, weather prediction
- **Example:** Machine translation (need to remember beginning of sentence)

GRU:

- **Middle ground:** Shorter than LSTM sequences but need memory
- **Faster than LSTM:** 25-30% fewer parameters
- **Similar performance:** Often matches LSTM with less computation
- **Example:** Speech recognition, music generation

Key Concepts:

1. Hidden State ($\mathbf{h_t}$):

- The "memory" of the network
- Carries information from previous timesteps
- Size determines capacity (common: 128, 256, 512)

2. return_sequences:

- `True`: Return output for every timestep [batch, seq_len, hidden]
- `False`: Return only last output [batch, hidden]
- **Use True** for: Sequence-to-sequence tasks
- **Use False** for: Classification from sequences

3. Bidirectional:

- Process sequence forward AND backward
- Captures context from both directions
- **Use for:** When entire sequence available (not streaming)
- Doubles parameters and computation

Practical example:

```
python
```

```

# Text classification (sentiment analysis)
Embedding(vocab_size=10000, embedding_dim=128) # Words → vectors
LSTM(128, return_sequences=False) # Process sequence, return final state
Dense(1, activation='sigmoid') # Binary classification

# Sequence-to-sequence (translation)
# Encoder
LSTM(256, return_sequences=True) # Process source language
LSTM(256, return_sequences=False) # Final encoder state

# Decoder
LSTM(256, return_sequences=True) # Generate target language
Dense(vocab_size, activation='softmax') # Predict words

```

Why LSTM over RNN?

- **Vanishing gradient problem:** Simple RNN gradients disappear over long sequences
- **LSTM gates:** Control information flow, preserve gradients
- **Long-term memory:** LSTM cell state acts as "highway" for information

Parameters comparison:

- RNN: $4 \times (\text{hidden_size} \times \text{input_size} + \text{hidden_size}^2)$
- LSTM: $4 \times \text{RNN}$ (4 gates)
- GRU: $3 \times \text{RNN}$ (3 gates) - 25% fewer than LSTM

5. Embedding Layer

Mathematical Definition:

Given integer input i (word index):
 $\text{embedding}[i] \rightarrow \mathbb{R}^d$ (d-dimensional vector)

Essentially a lookup table: $\text{matrix}[\text{vocab_size}, \text{embedding_dim}]$

What it does: Converts discrete tokens (words, items) into continuous dense vectors.

When to use:

- **Natural Language Processing:** Convert words to vectors
- **Recommendation systems:** Convert user/item IDs to embeddings
- **Categorical features:** Any discrete input with many categories

- **Graph neural networks:** Node embeddings

Why it works:

- **Semantic similarity:** Similar words get similar vectors
- **Dimensionality reduction:** 10,000 words $\rightarrow$ 300 dimensions
- **Learnable:** Embeddings trained specifically for your task

Key Concepts:

1. Vocabulary Size (**input_dim**):

- How many unique tokens (words, items)
- Example: 10,000 most common words

2. Embedding Dimension (**output_dim**):

- Vector size for each token
- Common choices:
 - 50-100: Small datasets, simple tasks
 - 128-256: Standard choice
 - 300-768: Large models (BERT uses 768)
- Trade-off: Larger = more expressiveness but more parameters

3. Pre-trained Embeddings:

- Word2Vec, GloVe, FastText: Trained on large corpora
- Transfer learning for NLP
- **Use when:** Limited training data
- **Fine-tune when:** Your domain is specific

Practical example:

```
python

# Movie recommendation
# 10,000 movies  $\rightarrow$  128-dim vectors
# 50,000 users  $\rightarrow$  128-dim vectors

movie_embedding = Embedding(10000, 128)
user_embedding = Embedding(50000, 128)

# Predict rating: dot product of embeddings
rating = user_vec · movie_vec

# Similar users/movies have high dot product
```

Why not one-hot encoding?

- One-hot: [0,0,0,1,0,...,0] (size = vocab_size)
 - 10,000 words = 10,000 dimensions
 - No similarity information
 - Sparse, inefficient
- Embedding: [0.2, -0.5, 0.8, ...] (size = embedding_dim)
 - 10,000 words = 128 dimensions (78x smaller)
 - Similar words have similar vectors
 - Dense, efficient

Parameters: $\text{vocab_size} \times \text{embedding_dim}$

- 10,000 words $\times$ 128 dim = 1.28M parameters
-

6. Normalization Layers

BatchNorm Mathematical Definition:

For each feature across batch:

$$\mu = (1/m) \sum x_i \quad (\text{mean})$$

$$\sigma^2 = (1/m) \sum (x_i - \mu)^2 \quad (\text{variance})$$

$$y = \gamma * (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$$

where:

- γ, β : learnable parameters
- ϵ : small constant for numerical stability (10^{-5})

What it does: Normalizes layer inputs to have mean=0, variance=1, then applies learnable scale and shift.

When to use:

BatchNorm:

- **Deep networks:** Essential for 10+ layers
- **Convolutional networks:** After conv layers (before or after activation)
- **When:** Batch size ≥ 16 (needs statistics from batch)
- **Not for:** Very small batches, online learning, or RNNs

LayerNorm:

- **RNNs/Transformers:** Batch size independent
- **Small batches:** Works with batch_size=1
- **Variable length sequences:** Better for NLP
- **Normalizes:** Across features, not batch

Why it works:

1. **Faster training:** Can use higher learning rates (10-100x)
2. **Reduces internal covariate shift:** Stabilizes input distributions
3. **Regularization effect:** Adds noise (each sample normalized differently)
4. **Gradient flow:** Prevents vanishing/exploding gradients

Practical example:

```
python

# CNN with BatchNorm
Conv2D(64, 3)
BatchNorm2D(64)  # Normalize 64 channels
ReLU()

# Why this order? Debated, but common:
# Conv → BN → Activation (most common)
# Conv → Activation → BN (also works)

# Without BatchNorm: may need learning_rate = 0.001
# With BatchNorm: can use learning_rate = 0.01 or higher
```

BatchNorm vs LayerNorm:

BatchNorm: Normalizes across batch dimension
 Input: [batch=32, channels=64, height=28, width=28]
 Normalizes: For each of 64 channels, across 32 samples

LayerNorm: Normalizes across feature dimension
 Input: [batch=32, features=512]
 Normalizes: For each sample, across 512 features

Training vs Inference:

- **Training:** Uses batch statistics (mean, variance)
- **Inference:** Uses running statistics (exponential moving average)
- **Why:** Single sample at inference, no batch to compute statistics

7. Dropout

Mathematical Definition:

Training:

$$y = x \odot m / (1-p)$$

where $m \sim \text{Bernoulli}(1-p)$ (random mask)

Inference:

$$y = x \text{ (no dropout)}$$

What it does: Randomly sets a fraction (p) of inputs to zero during training.

When to use:

- **Overfitting detected:** Training accuracy $\gg$ validation accuracy
- **After dense layers:** Most common placement
- **Large networks:** More parameters = more overfitting risk
- **Limited data:** Dropout acts as ensemble of sub-networks

When NOT to use:

- **Small models:** May hurt performance
- **Already underfitting:** Will make it worse
- **Convolutional layers:** Less common, use sparingly (dropout2d exists)

Why it works:

- **Prevents co-adaptation:** Neurons can't rely on specific other neurons
- **Ensemble effect:** Each training step uses different sub-network
- **Implicit regularization:** Similar effect to L2 regularization

Key Concepts:

1. Dropout Rate (p) :

- $p=0.2$: Drop 20% of neurons (light regularization)
- $p=0.5$: Drop 50% (standard choice, heavy regularization)
- $p=0.8$: Drop 80% (very aggressive, rarely used)

2. Placement:

```
python
```

```
# Standard pattern
```

```
Dense(512)
```

```
ReLU()
```

```
Dropout(0.5)    # After activation
```

```
Dense(256)
```

```
ReLU()
```

```
Dropout(0.5)
```

```
Dense(10)      # No dropout on output layer
```

3. Scaling:

- During training: multiply by $1/(1-p)$ to maintain expected output
- Ensures same scale during training and inference

Practical example:

```
python
```

```
# Without dropout: overfitting
```

```
# Train accuracy: 99%, Val accuracy: 75%
```

```
# With dropout: better generalization
```

```
# Train accuracy: 92%, Val accuracy: 88%
```

```
model = Sequential([  
    Dense(512, activation='relu'),  
    Dropout(0.5),    # Regularizes  
    Dense(256, activation='relu'),  
    Dropout(0.3),    # Less aggressive in deeper layers  
    Dense(10, activation='softmax')  
])
```

Activation Functions

Activation functions introduce non-linearity. Without them, stacked linear layers = single linear layer.

1. ReLU (Rectified Linear Unit)

Mathematical Definition:

$$f(x) = \max(0, x)$$

Derivative:

$$f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

When to use:

- **Default choice** for hidden layers
- **CNNs:** Almost always ReLU
- **MLPs:** Standard activation
- **Fast computation:** Just thresholding

Why it works:

- **Solves vanishing gradient:** Gradient is 1 for positive values
- **Sparse activations:** Many neurons output 0 (efficient)
- **Biological plausibility:** Neurons either fire or don't

Problems:

- **Dying ReLU:** If neuron outputs always negative, gradient = 0 forever
 - Solution: Use LeakyReLU or careful initialization

2. LeakyReLU

Mathematical Definition:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \text{ (typically } \alpha = 0.01) \end{cases}$$

When to use:

- **Dying ReLU problem:** When many neurons become inactive
- **Alternative to ReLU:** Similar benefits, prevents death
- **Experimentation:** Try if ReLU doesn't work well

3. Sigmoid

Mathematical Definition:

$$f(x) = 1 / (1 + e^{(-x)})$$

Output range: (0, 1)

When to use:

- **Binary classification:** Output layer (single neuron)
- **Probabilities:** When need output in [0,1]
- **Gates in LSTM/GRU:** Controls information flow

When NOT to use:

- **Hidden layers:** Causes vanishing gradient problem
- **Multi-class:** Use softmax instead

Why output layer only:

- Gradients very small for $|x| > 3$
- Network learns slowly in hidden layers

4. Tanh

Mathematical Definition:

$$f(x) = (e^x - e^{(-x)}) / (e^x + e^{(-x)})$$

Output range: (-1, 1)

When to use:

- **RNN hidden states:** Traditionally used (before LSTM/GRU)
- **When need negative outputs:** Unlike sigmoid [0,1]
- **Centered data:** Mean closer to 0 than sigmoid

Advantage over sigmoid:

- Zero-centered: Easier optimization
- Stronger gradients: Range [-1, 1] vs [0, 1]

5. Softmax

Mathematical Definition:

For vector $x = [x_1, \dots, x_n]$:

$$\text{softmax}(x)_i = e^{(x_i)} / \sum_j e^{(x_j)}$$

Properties:

- Output sums to 1
- Each output in (0, 1)
- Represents probability distribution

When to use:

- **Multi-class classification:** Always use for output layer
- **Probability distribution:** When need valid probabilities
- **Mutually exclusive classes:** Cat OR dog (not both)

Why it works:

- Converts logits (raw scores) to probabilities
- Differentiable: Can backpropagate through it
- Emphasizes largest values (exponential)

Practical example:

```
python

# 10-class classification
logits = [2.0, 1.0, 0.1, ...] # Raw network outputs

# Softmax
probs = softmax(logits)
# [0.68, 0.25, 0.03, ...] # Probabilities sum to 1

predicted_class = argmax(probs) # Class with highest probability
```

Loss Functions

Loss functions measure how wrong your predictions are. Training minimizes this loss.

1. Cross-Entropy Loss

Mathematical Definition:

Binary Cross-Entropy:

$$L = -[y * \log(\hat{y}) + (1-y) * \log(1-\hat{y})]$$

where:

- y: true label (0 or 1)
- $\hat{y}$: predicted probability

Categorical Cross-Entropy:

$$L = -\sum_i y_i * \log(\hat{y}_i)$$

where:

- y_i : true probability (one-hot: [0,0,1,0,...])
- $\hat{y}_i$: predicted probability from softmax

When to use:

- **Classification tasks:** Primary choice
- **Binary:** Sigmoid output + BCE loss
- **Multi-class:** Softmax output + CE loss
- **Probabilistic outputs:** When predictions are probabilities

Why it works:

- **Information theory:** Measures difference between distributions
- **Probabilistic interpretation:** Maximum likelihood estimation
- **Gradient behavior:** Large errors give large gradients (fast learning)

Practical example:

```
python
```

```
# Binary: Spam detection (0=not spam, 1=spam)
```

```
true = 1
```

```
predicted = 0.8 # Model 80% confident it's spam
```

```
loss = -(1*log(0.8)) = 0.22 # Small loss (correct)
```

```
predicted = 0.2 # Model only 20% confident
```

```
loss = -(1*log(0.2)) = 1.61 # Large loss (wrong)
```

```
# Multi-class: Image classification
```

```
true = [0, 0, 1, 0, 0] # Class 2
```

```
predicted = [0.1, 0.2, 0.6, 0.05, 0.05]
```

```
loss = -(0*log(0.1) + 0*log(0.2) + 1*log(0.6) + ...)
      = -log(0.6) = 0.51
```

2. Mean Squared Error (MSE)

Mathematical Definition:

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

When to use:

- **Regression tasks:** Predicting continuous values
- **Price prediction:** House prices, stock prices
- **Temperature, age, salary:** Any numerical output
- **When outliers matter:** MSE heavily penalizes large errors

Why it works:

- **Least squares:** Optimal under Gaussian noise assumption
- **Convex:** Single global minimum (easy optimization)
- **Interpretable:** Same units as target (squared)

Practical example:

```
python
```

House price prediction

true = 300,000

predicted = 320,000

error = 20,000

MSE = $20,000^2 = 400,000,000$

Temperature prediction

true = 25°C

predicted = 24°C

error = 1°C

MSE = $1^2 = 1$

3. Mean Absolute Error (MAE)

Mathematical Definition:

$$\text{MAE} = (1/n) \sum |y_i - \hat{y}_i|$$

When to use:

- **Outlier-robust regression:** When outliers shouldn't dominate
- **Interpretable errors:** Errors in same units as target
- **When all errors equal:** Don't want to heavily penalize large errors

MSE vs MAE:

- **MSE:** Penalizes large errors more (quadratic)
 - Use when: Large errors are particularly bad
- **MAE:** Treats all errors linearly
 - Use when: All errors equally important

Example:

python

Errors: [1, 2, 10]

$$\text{MAE} = (1 + 2 + 10) / 3 = 4.33$$

$$\text{MSE} = (1^2 + 2^2 + 10^2) / 3 = (1 + 4 + 100) / 3 = 35$$

MSE dominated by outlier (10), MAE treats all fairly

4. Huber Loss

Mathematical Definition:

$$L = 0.5 * (y - \hat{y})^2 \quad \text{if } |y - \hat{y}| \leq \delta$$
$$\delta * (|y - \hat{y}| - 0.5\delta) \quad \text{otherwise}$$

When to use:

- **Best of both worlds:** MSE for small errors, MAE for large
 - **Robust regression:** When some outliers exist
 - **Parameter δ :** Controls transition point (typically 1.0)
-

Optimizers

Optimizers update network weights to minimize loss. They implement gradient descent variants.

Basic Gradient Descent

Mathematical Definition:

$$\theta_{t+1} = \theta_t - \eta * \nabla L(\theta_t)$$

where:

- θ : parameters (weights)
- η : learning rate
- ∇L : gradient of loss

Problem: Updates all parameters with same learning rate. Slow convergence.

1. SGD (Stochastic Gradient Descent)

Mathematical Definition:

$$v_t = \gamma * v_{t-1} + \eta * \nabla L(\theta_t) \quad (\text{momentum})$$

$$\theta_{t+1} = \theta_t - v_t$$

where:

- γ : momentum (typically 0.9)
- Accumulates velocity in consistent directions

When to use:

- **Simple problems:** Small networks, simple data

- **Well-understood data:** When good hyperparameters known
- **Momentum helps:** Gets stuck in local minima less

Why momentum:

- Accelerates in consistent directions
- Dampens oscillations
- Escapes shallow local minima

Practical example:

```
python

# Without momentum: oscillates in ravines
# With momentum: accelerates downhill

optimizer = SGD(lr=0.01, momentum=0.9)
```

2. Adam (Adaptive Moment Estimation)

Mathematical Definition:

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla L \quad (1st \text{ moment: mean})$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * \nabla L^2 \quad (2nd \text{ moment: variance})$$

$$\hat{m}_t = m_t / (1 - \beta_1^t) \quad (\text{bias correction})$$

$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

$$\theta_{t+1} = \theta_t - \eta * \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

Defaults: $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$

When to use:

- **Default choice:** Works well for most problems
- **Deep networks:** Adapts learning rate per parameter
- **Sparse gradients:** NLP, embeddings
- **Start here:** If it doesn't work, try others

Why it works:

- **Adaptive learning rates:** Different rate for each parameter
- **Momentum:** Accelerates learning

- **Moving average of gradients:** Smooths noisy estimates
- **Parameter-wise:** Handles different scales automatically

Key Concepts:

1. Learning Rate (η):

- Default: 0.001 (works well often)
- Too high: Unstable, loss explodes
- Too low: Slow convergence
- Start with 0.001, decrease if unstable

2. β_1, β_2 :

- Rarely need to change
- β_1 controls momentum
- β_2 controls adaptive learning rate

Practical example:

```
python

# Parameter 1: gradients = [1, 1, 1, 1, ...] (consistent)
# → Adam increases learning rate for this parameter

# Parameter 2: gradients = [1, -1, 1, -1, ...] (noisy)
# → Adam decreases learning rate for this parameter

# Automatically handles different parameters!
```

3. AdamW

Mathematical Definition:

Same as Adam, but weight decay applied directly:

$$\theta_{t+1} = \theta_t - \eta * (\hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda * \theta_t)$$

where λ is weight decay coefficient

When to use: